

Study Spot Finder

Design Document

Authors	Eleah Burman and Bryan James
Class	CS5610 Web Development, Northeastern University
Live site	https://study-spot-finder.onrender.com
Repository	https://github.com/BryanSJamesDev/study-spot-finder
Figma	Full Figma file (desktop and mobile)

Contents

1. Project Description
2. User Personas
3. User Stories
4. Design Mockups
5. Design Decisions
6. Architecture
7. Data Model
8. Algorithms
9. Team Contributions

1. Project Description

The problem

Finding a good place to study on campus usually means walking from building to building until you find an open seat. Even then, the spot might be too loud, have no outlets, or close an hour after you sit down. Students rely on word of mouth or luck, and the information they need (how quiet is it, can I plug in my laptop, is it packed right now) isn't written down anywhere.

Our solution

Study Spot Finder is a web app where Northeastern students share and find study spots on campus. Students can:

- **Add and browse study spots** with noise level, outlets, Wi-Fi, closing time, and tags such as “group study” or “natural light.”
- **See spots on a campus map** and filter them by name, building, noise, tags, outlets, and Wi-Fi.
- **Rate spots** from 1 to 5 stars and see each spot’s average rating.
- **Check in** to report how crowded a spot is, and see how busy it is right now based on the last 2 hours of check-ins.
- **Get recommendations** for similar spots from a k-nearest neighbors algorithm that explains why each spot matched.
- **See a crowd forecast** for the next 6 hours, based on past check-ins.

Who it’s for

Northeastern students who study on campus, especially those with specific needs, like a quiet room, an outlet, or a table for a group, who don’t want to waste time searching.

Why it’s useful

The app turns scattered, word-of-mouth knowledge into shared, searchable information. Because students add the spots, ratings, and check-ins themselves, the information stays current, and the more people use it, the better the recommendations and forecasts get.

2. User Personas

Maya, the quiet-study grad student	
Who	26, first-year Master's student in Data Science. Commutes from Brooklyn three days a week.
Habits	Comes to campus with a long block of time between classes. Needs a silent space and an outlet, since her laptop battery dies by midday.
Frustrations	Wastes 20 minutes walking between buildings looking for a quiet spot with an open outlet. The library's silent floor is often full by the time she arrives.
Needs from the app	Filter by noise level and outlets, and see how crowded a spot is before she walks over.

Jordan, the group-project organizer	
Who	20, third-year undergrad in Business. Always the one coordinating study sessions for group projects.
Habits	Needs a table for 4 to 6 people where talking is okay. Plans sessions ahead.
Frustrations	Group study rooms are booked up, and open tables are hard to find at peak times. He doesn't know when spots are least busy.
Needs from the app	Tags like "group study," the crowd forecast to pick a time when tables will be open, and ratings from other students.

Priya, the new transfer student	
Who	19, second-year undergrad who just transferred to Northeastern. Doesn't know the campus yet.
Habits	Studies between classes in whatever building she's already in. Likes natural light and a calm, but not silent, atmosphere.
Frustrations	Only knows the main library. Doesn't know which other buildings have good spaces or where they are.
Needs from the app	The campus map to discover spots near her, and "Spots like this" to find new places similar to one she already likes.

3. User Stories

Story 1: Adding a study spot (Eleah)

> *As a student, I want to add, edit, and delete a study spot with its noise level, outlets, Wi-Fi, closing time, tags, and location, so that other students know what it's like.*

Priya finds a bright, calm lounge on the second floor of a building near her classes. It has big windows and a few outlets along the wall. She opens the Spots page and fills out the "Add a study spot" form: name and building, noise level Moderate, outlets and Wi-Fi available, closing time, and the tag "natural light." She clicks the map to drop a pin where the lounge is, and the spot appears on the map and in the list right away. A week later, the lounge starts closing earlier, so she clicks Edit and updates the closing time.

Story 2: Finding a spot on the map, and similar ones (Eleah)

> *As a student, I want to see spots on a map, filter them by noise, outlets, and tags, and see a "Spots like this" list on any spot, so that I can quickly find a place that fits how I study.*

Maya has three hours before her evening class. On the Spots page, she chooses Quiet and checks "Has outlets." The cards and map pins narrow down as she filters. The library is usually packed by noon, so she clicks "Spots like this" on its card. It shows three quiet spots in other buildings, each with a reason, like "Same noise level · Both have outlets · Both have Wi-Fi · 1 shared tag." She clicks one to jump to its card and heads there instead.

Story 3: Rating a study spot (Bryan)

> *As a student, I want to give a spot a 1 to 5 star rating, edit or delete my rating, and see each spot's average rating, so that I know which places are worth going to.*

Jordan's group met at the student center. The tables were big enough, but it got loud around lunch. After the session, he opens the Ratings page, picks the student center, and gives it 3 stars. The average updates. The next time he plans a session, he checks Campus favorites first and picks a top-rated spot.

Story 4: Checking in and seeing the crowd forecast (Bryan)

> *As a student, I want to submit, edit, and delete a check-in that reports how crowded a spot is, see how crowded it is right now, and see how busy it's predicted to be over the next few hours, so that I can avoid crowded times.*

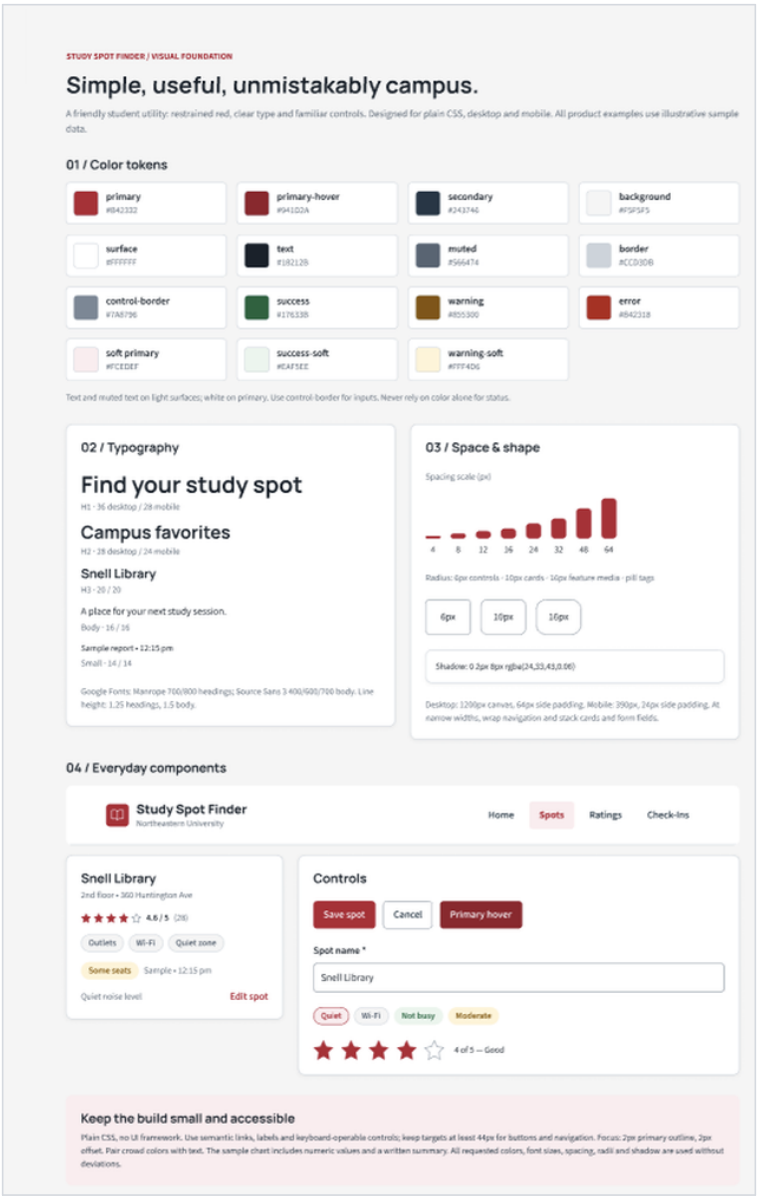
Jordan needs a group session later today. On the Check-Ins page, he picks a spot in Forecast and sees it's busy now but quieter in a few hours, so he schedules the session for later. When the group arrives, he submits a check-in: "Not busy," with a note that the big tables by the window are open. Maya, across campus, sees the spot under "Crowded right now" and heads over too.

4. Design Mockups

Mockups were designed in Figma for desktop and mobile. View the full Figma file. Each page below shows the mockup, followed by what changed in the final build.

Style guide

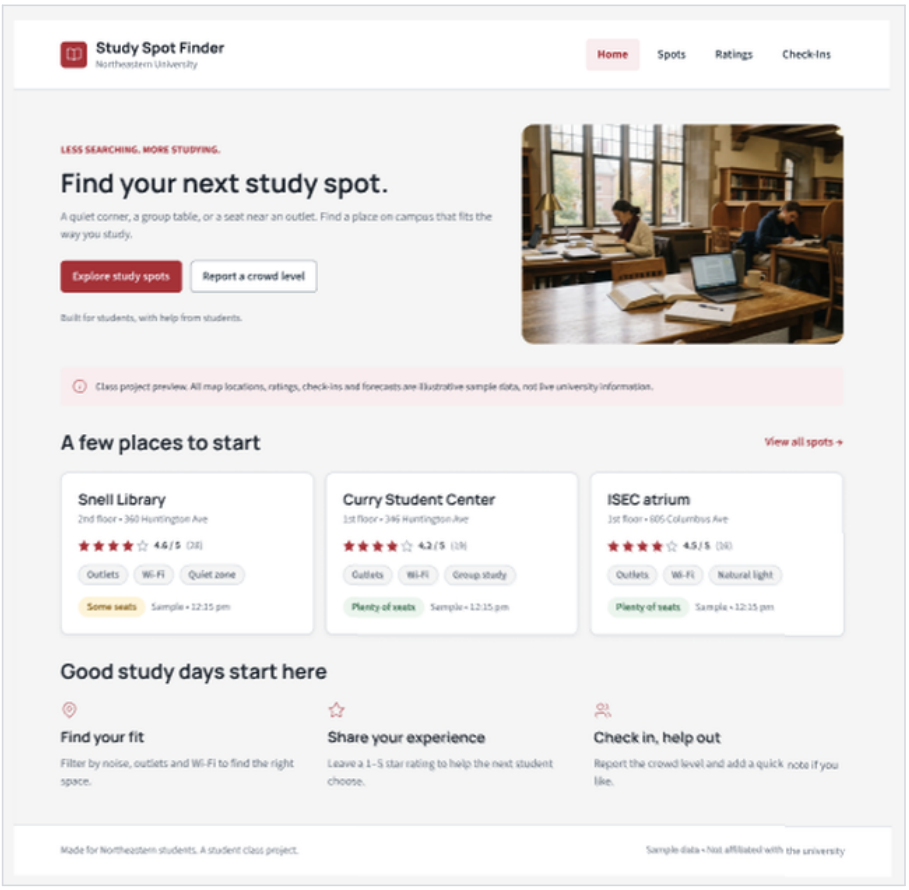
Defines the color tokens, typography, spacing scale, and shared components (nav, cards, buttons, form inputs, tags, and star rating). These map directly to the CSS variables in `public/css/shared.css`.



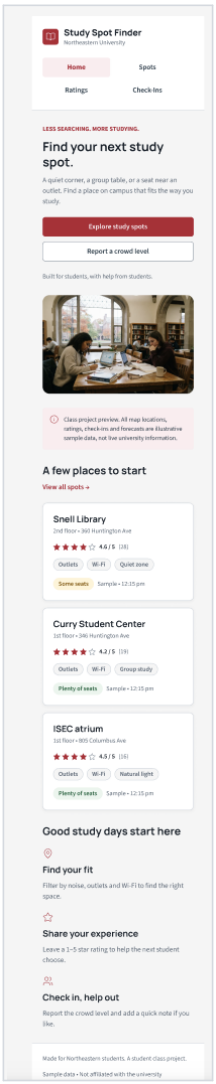
Style guide: color tokens, typography, spacing, and components

Home

Introduces the app with a headline and two main actions: “Explore study spots” and “Report a crowd level.” Below that, featured spot cards and a three-step summary: find a spot, rate it, and check in.



Home: desktop



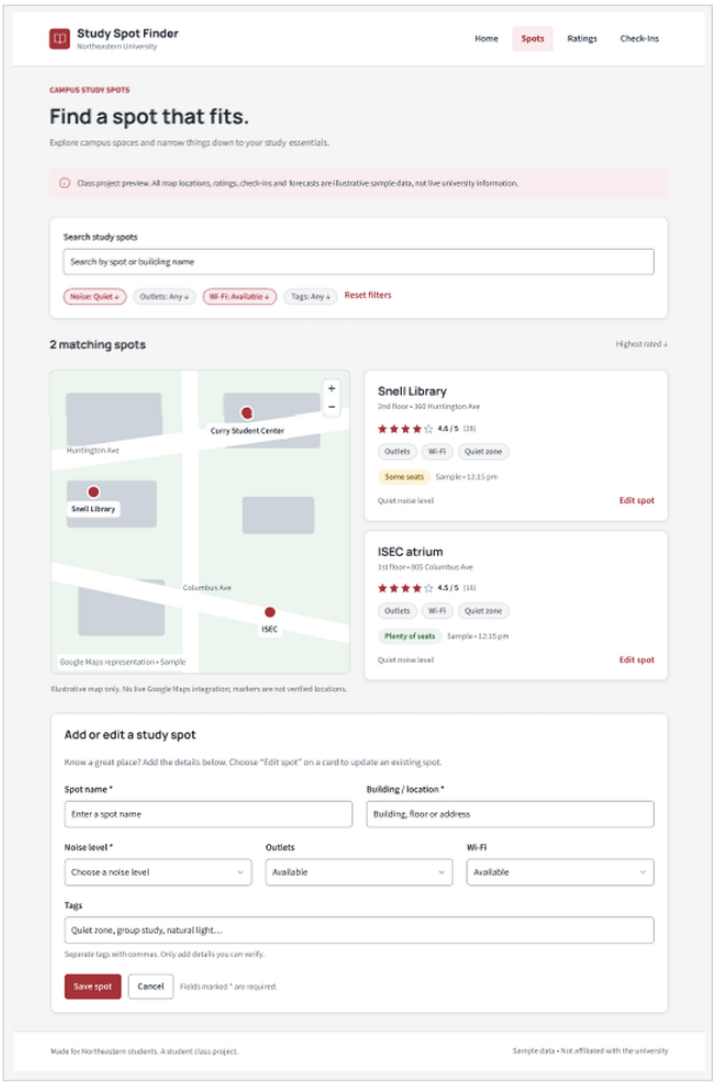
Mobile

Changes in the final build

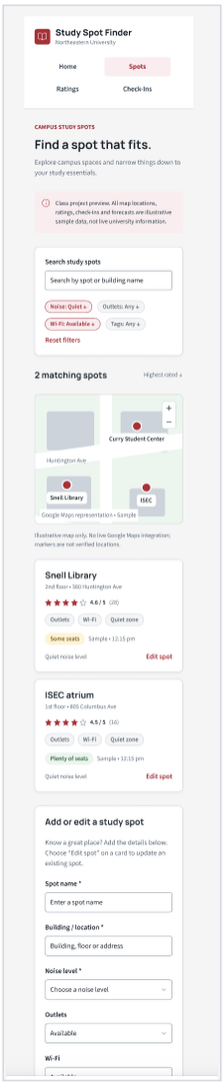
- Built as designed, with a shared nav on every page.

Spots

The main page for finding a spot: search and filters above a map and a list of spot cards, with the add or edit form at the bottom.



Spots: desktop



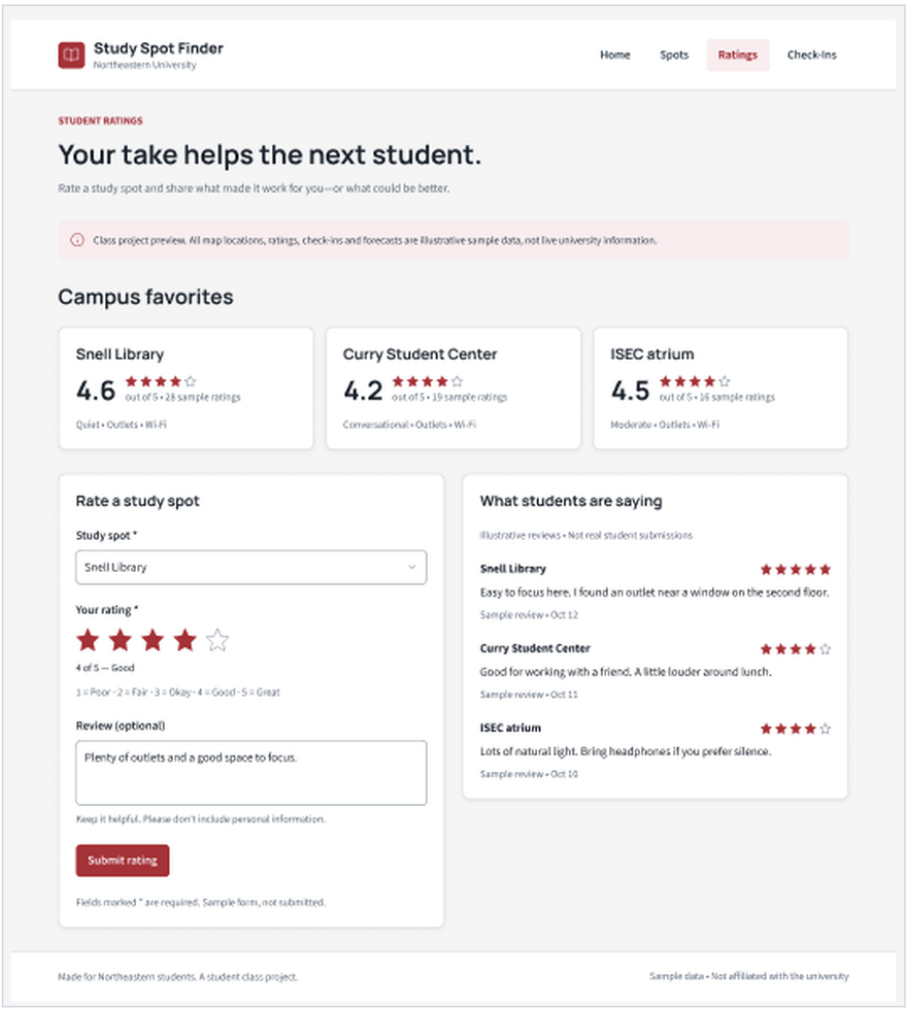
Mobile

Changes in the final build

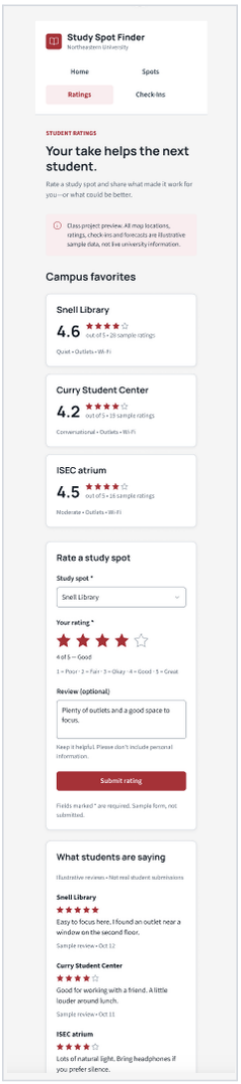
- Filters became a compact form: search on top, then noise level, tag, outlets, Wi-Fi, and Clear filters in one row.
- The map sits next to the list on desktop and stays in view while scrolling. It uses Leaflet and OpenStreetMap, limited to campus.
- The form gained a closing time field and a location set by clicking the map. The server checks that the location is on campus.
- Each card gained a “Spots like this” button that shows the 3 most similar spots (KNN) with reasons.
- Deleting asks for confirmation in a styled dialog that every page shares, instead of the browser’s confirm().

Ratings

Campus favorites shows the top-rated spots. A form lets students pick a spot and choose 1 to 5 stars, and recent ratings are listed with a spot filter.



Ratings: desktop



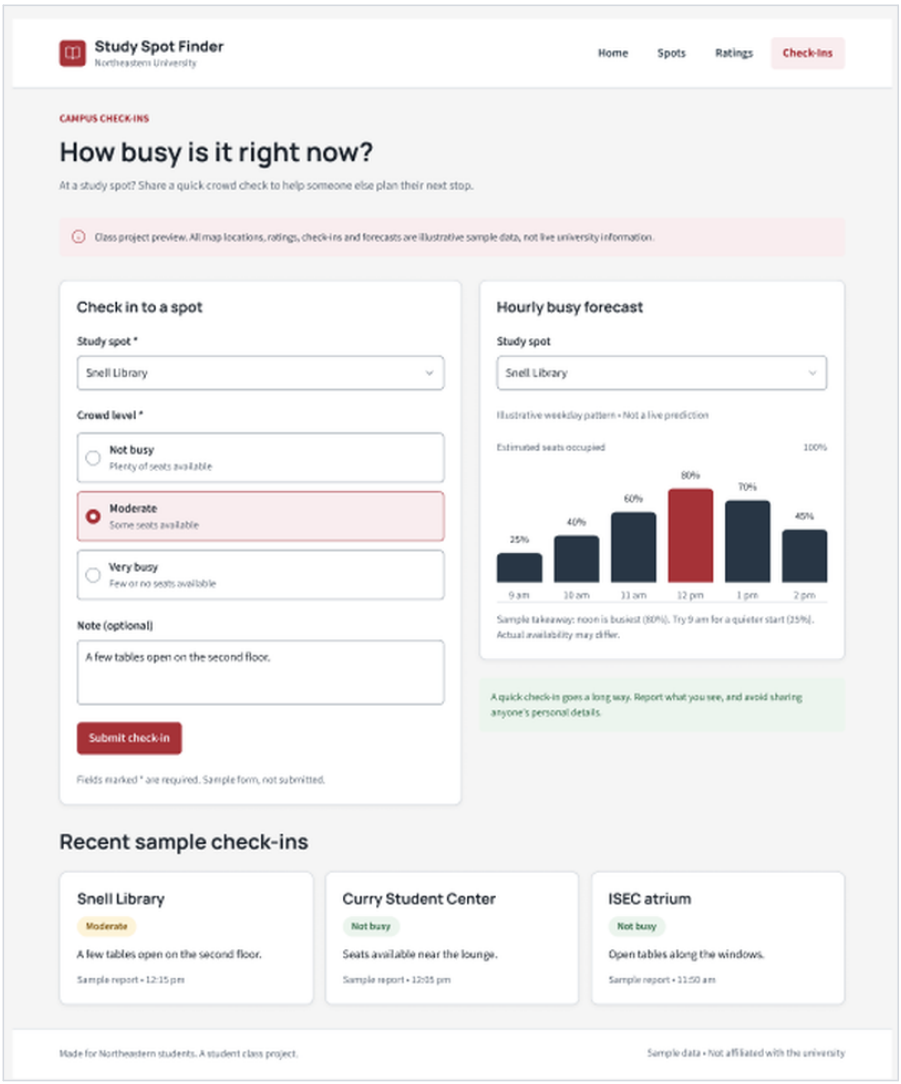
Mobile

Changes in the final build

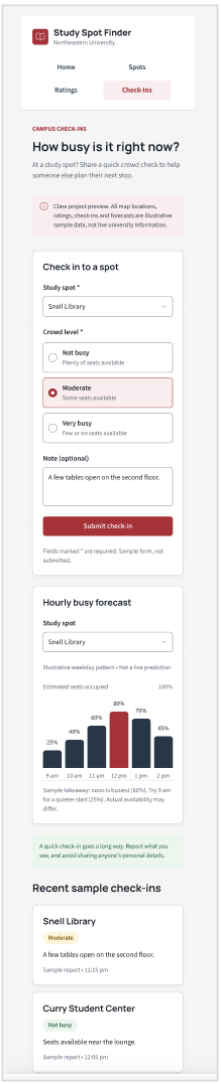
- Ratings are stars only. We dropped the optional written review to keep rating fast, so “What students are saying” became a recent ratings list with a filter.
- Campus favorites shows the top 3, with the form right below so it’s easy to find.

Check-Ins

A form for reporting how crowded a spot is (Not busy, Moderate, or Very busy) with an optional note, plus a crowd forecast and recent check-ins.



Check-Ins: desktop



Mobile

Changes in the final build

- The hourly bar chart became 6 forecast cards for the next 6 hours, each with a crowd badge and a confidence level (low, medium, or high).
- Hours without enough data say “Not enough check-ins to forecast” instead of guessing.
- A “Crowded right now” panel averages check-ins from the last 2 hours.

Mobile

On narrow screens, the nav wraps into a grid, and side-by-side sections (like the map and spot list) stack into one column. Buttons and nav links stay at least 44px tall so they’re easy to tap.

5. Design Decisions

Visual style

- **Northeastern red.** The primary color (#b42332) nods to the school colors but is used sparingly: main buttons, the active nav link, and highlights. Most of the page is white panels on light gray, so the information stays the focus.
- **Two fonts.** Manrope for headings feels friendly and modern. Source Sans 3 for body text is easy to read at small sizes.
- **Cards for scanning.** Every spot card has the same layout (name, building, noise and closing time, tags), so students can compare at a glance.
- **Design tokens.** Colors, font sizes, spacing, and corner radii are CSS variables in shared.css, so one change updates the whole app.
- **Favicon.** A white map pin on the app's red, shared by every page.

Accessibility

- **Color contrast** meets WCAG AA for text.
- **Color is never the only signal.** Crowd levels always show text ("Not busy," "Moderate"), and ratings show the number ("4.6 out of 5").
- **Standard HTML.** Buttons are <button>, filters are a real <form> with <select> and checkboxes, the star rating is a <fieldset> of radio buttons, and the delete confirmation is a native <dialog>. All of it works with a keyboard and screen readers.
- **Visible focus** on every interactive element. The delete dialog focuses Cancel first, so pressing Enter can't delete by accident, and Escape closes it.
- **Large tap targets** of at least 44px.
- **Reduced motion respected.** Smooth scrolling turns off for people who choose reduced motion.
- **ARIA where it helps:** aria-current="page" on the active nav link, and labels on buttons like "Edit Snell Quiet Floor."

Product decisions

- **Students add the data,** so it stays current without anyone maintaining it.
- **Simple crowd levels.** Three levels are quick to report, so check-ins actually happen.
- **Explain recommendations.** "Spots like this" says why each spot matched, so students can trust it.
- **Free, keyless map.** Leaflet with OpenStreetMap, at the professor's suggestion: no usage charges and no key to protect.
- **Simulated users.** The professor confirmed a login system isn't needed, so the app focuses on study spot features.

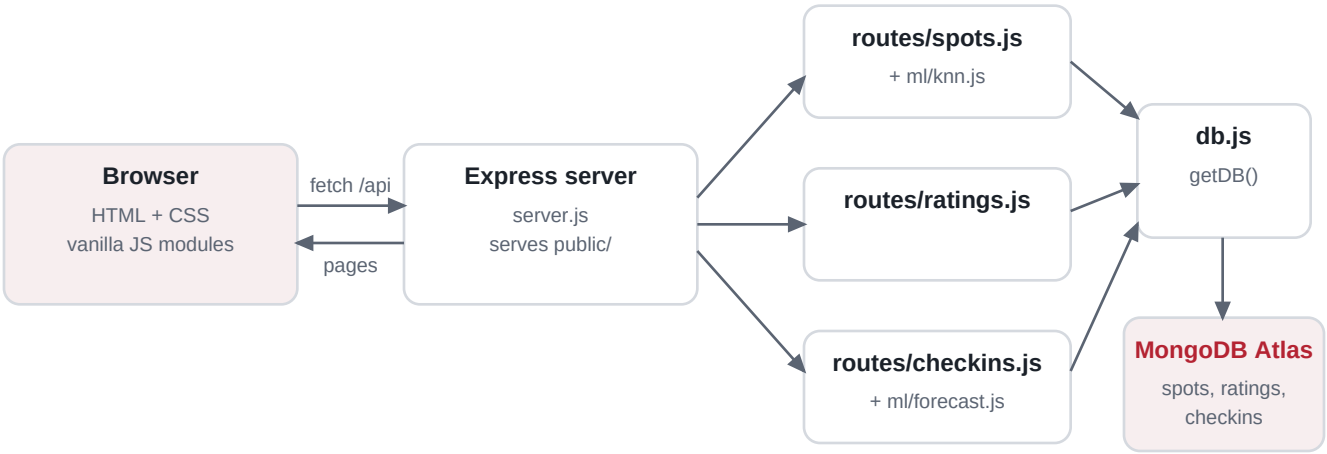
6. Architecture

Stack

Layer	Technology
Frontend	HTML5, CSS split by module, vanilla JavaScript (ES modules)
Backend	Node.js and Express
Database	MongoDB Atlas, using the official MongoDB Node.js driver
Map	Leaflet with OpenStreetMap (free, no API key)
Tooling	ESLint, Prettier, and Node's built-in test runner
Hosting	Render

No React, Mongoose, template engines, or server-side rendering. All JavaScript uses ES modules (import and export), never require().

How the pieces connect



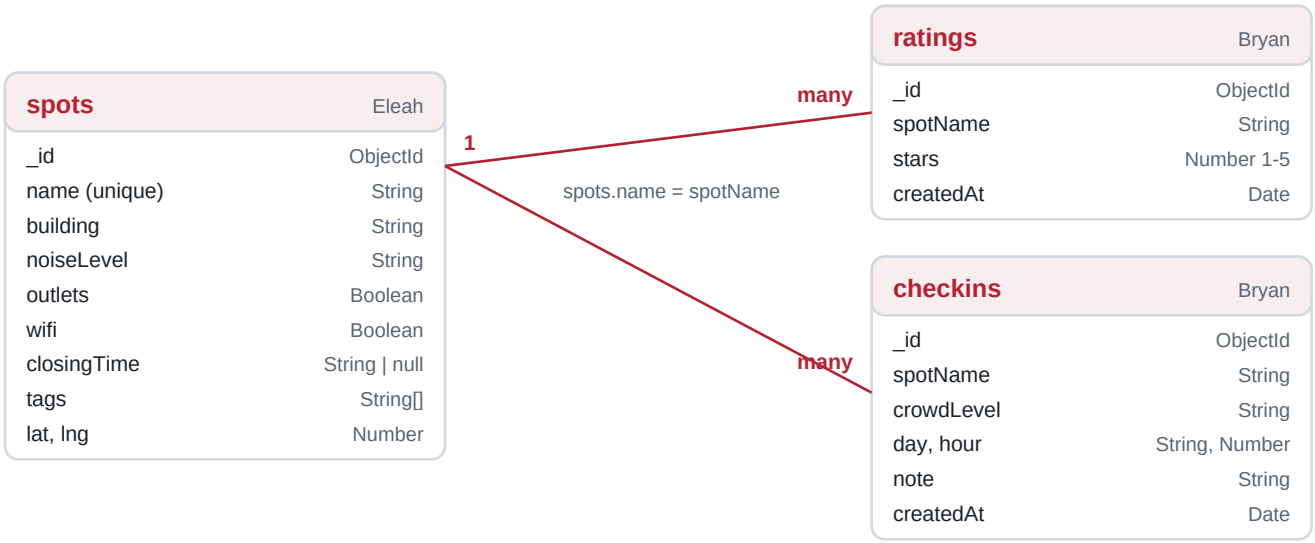
1. **Express serves the frontend.** server.js serves the public folder (HTML, CSS, and browser JavaScript) as static files.
2. **The browser renders the page.** Each page starts as a mostly empty shell, and its JavaScript module fetches data from the API and builds the content. This is client-side rendering, with no template engine.
3. **Routes handle the data.** Each collection has its own route file mounted under /api, like routes/spots.js for /api/spots.
4. **One shared database connection.** db.js connects once at startup, and every route calls getDB() to reuse it.
5. **The server starts only after the database connects,** so no request reaches a route before the database is ready.

Code organization

- **Backend:** server.js, db.js, routes/ (one file per collection), ml/ (KNN and forecast), and seed/ (test data).
- **Frontend:** one HTML file and JS module per page (home, spots, ratings, check-ins), plus shared confirmDialog.js. CSS split by module (shared.css, nav.css, home.css, one per page).
- **Tests:** tests/, run with npm test. Route tests send real requests to a test server and clean up their data.
- **Security:** the MongoDB connection string lives in a gitignored .env file and is never pushed, and the map needs no key.

7. Data Model

Three MongoDB collections, each with full CRUD through its own API routes. Ratings and check-ins point to a spot by its unique name.



Entity relationship diagram: one spot has many ratings and many check-ins

spots (Eleah)

Field	Type	Notes
_id	ObjectId	Created by MongoDB
name	String	Required, unique, trimmed
building	String	Required, trimmed
noiseLevel	String	Required: Quiet, Moderate, or Loud
outlets, wifi	Boolean	Optional, default false
closingTime	String	Optional 24-hour “HH:MM,” or null
tags	Array of strings	Optional, default []
lat, lng	Number	Optional, must be on campus, or both null

ratings (Bryan)

Field	Type	Notes
_id	ObjectId	Created by MongoDB
spotName	String	Required, matches a spot’s name
stars	Number	Required, whole number from 1 to 5
createdAt	Date	Set by the server

checkins (Bryan)

Field	Type	Notes
_id	ObjectId	Created by MongoDB
spotName	String	Required, matches a spot’s name
crowdLevel	String	Not busy, Moderate, or Very busy
day, hour	String, Number	Set by the server in campus time, used by the forecast
note	String	Optional, up to 200 characters
createdAt	Date	Set by the server, used for “Crowded right now”

Decisions

- **Reference spots by name,** so each teammate’s code stays independent. The tradeoff is that names must be unique, so the API rejects duplicates with a 409.
- **Renamed or deleted spots** leave their ratings and check-ins as they are.
- **Errors** return { "error": "message" } with status 400, 404, 409, or 500.
- **Test data:** seed scripts create 20 spots, 400 ratings, and 600 check-ins, for 1,020 records. They’re safe to re-run and never touch data added through the app.

8. Algorithms

Both algorithms are written from scratch in plain JavaScript, with no libraries.

KNN recommender: “Spots like this” (Eleah)

Goal: when a student looks at a spot, show the 3 most similar spots and explain why each one matched. Available at GET /api/spots/:id/similar and through the “Spots like this” button on every card.

1. **Turn each spot into numbers:** noise (Quiet 0, Moderate 0.5, Loud 1), outlets (1 or 0), Wi-Fi (1 or 0), and tags.
2. **Compare tags** with Jaccard similarity: shared tags divided by all unique tags, ignoring case, spaces, and duplicates.
3. **Measure distance** with Euclidean distance across the four features. 0 means identical.
4. **Pick the k closest** (k = 3), never including the spot itself, with ties broken by name.
5. **Explain the match,** like “Same noise level · Both have outlets · 1 shared tag.”

```
distance = sqrt( noise^2 + outlets^2 + wifi^2 + (1 - tagSimilarity)^2 )
```

Crowded right now (Bryan)

Averages check-ins from the last 2 hours (Not busy 1, Moderate 2, Very busy 3) for each spot. With no recent check-ins, it says so instead of guessing.

Crowd forecast (Bryan)

Goal: predict how busy a spot will be over the next 6 hours.

1. **Gather past check-ins** for the spot from the last 90 days.
2. **Weight each check-in** by day (same day counts fully, same kind of day counts less), hour (halves for each hour away, up to 2 hours), and recency (halves every 14 days).
3. **Take the weighted average** of crowd scores and round it back to a crowd level.
4. **Add a confidence level** (low, medium, or high) based on how much matching data there is. With none, it says “Not enough check-ins to forecast.”
5. **Show the next 6 hours** as cards, starting at the current campus hour and rolling past midnight.

Why weighted: a simple average would treat a Monday 9 a.m. check-in the same as a Friday 3 p.m. one. Weighting by day, hour, and recency reflects the real pattern for that time while still using nearby data.

9. Team Contributions

Eleah Burman

- Spots: page, form, and routes/spots.js with full CRUD and location validation
- Leaflet map, filters, and the spots seed data
- KNN recommender and “Spots like this”
- db.js, server.js, and all CSS by module
- Home page, shared nav, delete dialog, and favicon
- Figma mockups, style guide, README, and this document
- ESLint, Prettier, MIT license, and deployment on Render

Bryan James

- Ratings: page and routes/ratings.js with full CRUD and averages
- Check-ins: page and routes/checkins.js with full CRUD and Crowded right now
- Crowd forecast algorithm and its page section
- Seed scripts for 400 ratings and 600 check-ins
- GitHub repository and MongoDB Atlas setup

Shared: data model decisions, code reviews on every pull request, the demo video, and the presentation.

10. API, Setup, and Running Locally

The details below match the README.

Deployed site: https://study-spot-finder.onrender.com

API endpoints

Method	Endpoint	Owner	Notes
GET, POST	`/api/spots`	Eleah	
GET, PUT, DELETE	`/api/spots/:id`	Eleah	
GET	`/api/spots/:id/similar`	Eleah	The 3 most similar spots (KNN), each with a distance and reasons
GET, POST	`/api/ratings`	Bryan	GET accepts an optional ?spotName=
GET	`/api/ratings/averages`	Bryan	Average stars and count for each spot
GET, PUT, DELETE	`/api/ratings/:id`	Bryan	
GET, POST	`/api/checkins`	Bryan	GET accepts optional ?spotName= and ?limit= (1 to 100)
GET	`/api/checkins/crowded-now`	Bryan	Average crowd level for each spot from the last 2 hours
GET	`/api/checkins/forecast`	Bryan	Needs ?spotName=, optional ?hours= (1 to 24, default 6)
GET, PUT, DELETE	`/api/checkins/:id`	Bryan	

Errors are returned as { "error": "message" } with status 400 (bad input), 404 (not found), 409 (duplicate), or 500 (server error).

Scripts

Command	What it does
`npm run dev`	Starts the server and restarts on save
`npm start`	Starts the server (used for deployment)
`npm run lint`	Checks for code errors with ESLint
`npm run format`	Formats all files with Prettier
`npm run format:check`	Checks formatting without changing files
`npm test`	Runs tests with Node's built-in test runner

Environment variables

Each person makes their own .env file in the project root. It is gitignored and never pushed.

```
MONGO_URI=your-connection-string-here
PORT=3000
```

The database name is studySpotFinder. The map uses Leaflet with OpenStreetMap, which needs no API key.

Running locally

```
npm install
npm run dev
```

Then open http://localhost:3000. To fill the database with sample data:

```
npm run seed:spots
npm run seed:ratings
npm run seed:checkins
```

Seed scripts are safe to run more than once. "Crowded now" only counts check-ins from the last 2 hours, so run npm run seed:checkins again right before a demo.

License

MIT (see the LICENSE file).